

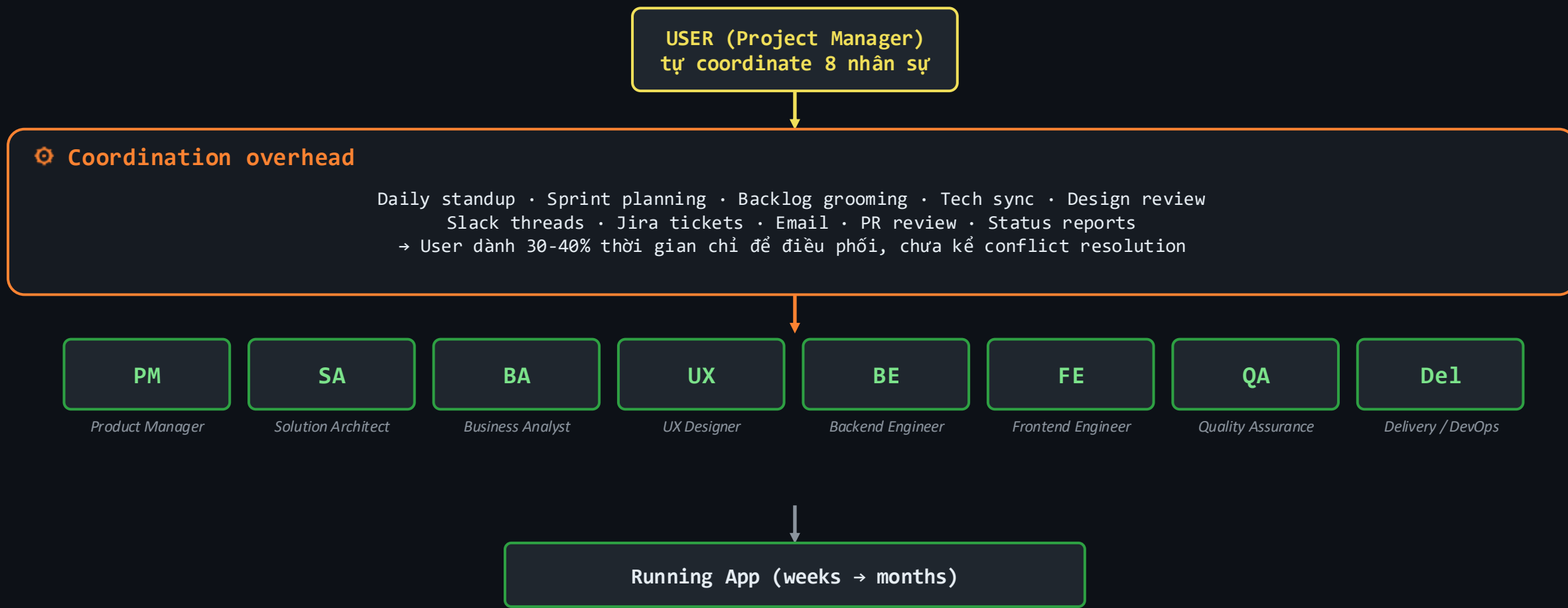
```
$ product-engineering-template --start
```

PRODUCT ENGINEERING AGENT FRAMEWORK

*Một đội kỹ sư sản phẩm AI trên tmux — 9 vai trò, 9 pane, 9 model.
Học theo nguyên lý BMAD. Talk với Orchestrator, để cả đội cùng làm việc.*

Trước khi có AI: team 8 nhân sự, user là PM

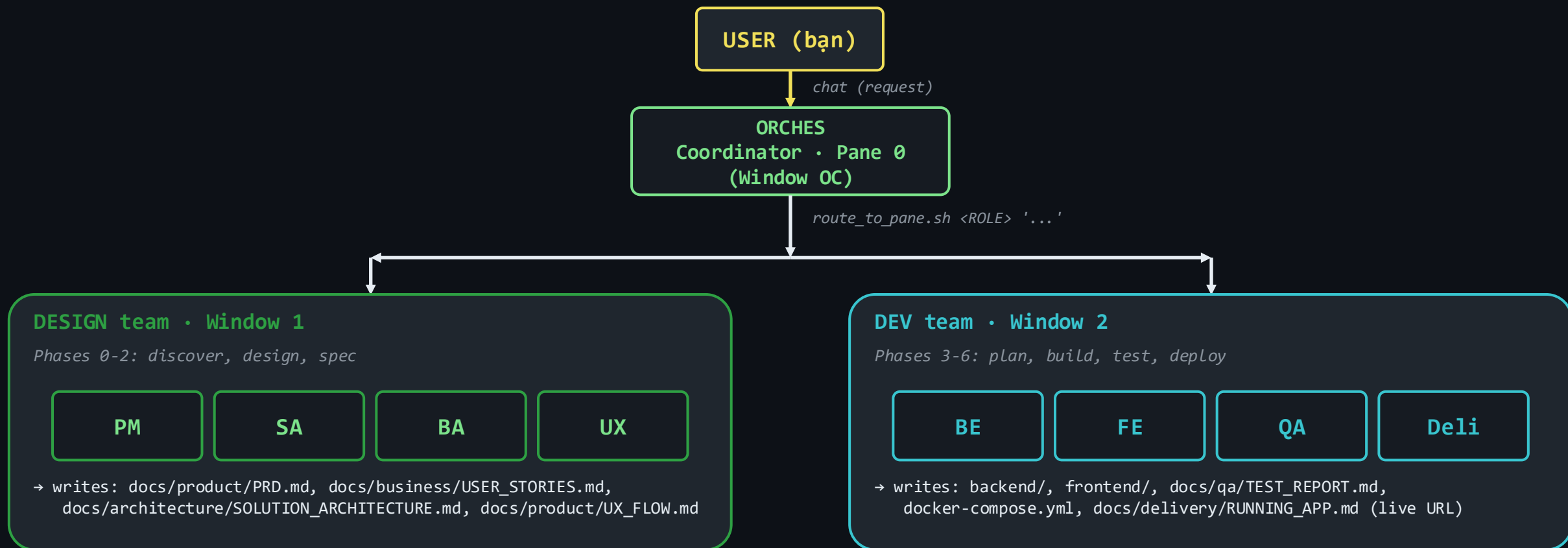
Workflow truyền thống — user phải tự coordinate, meetings overhead, weeks-to-months timeline



→ Slide tiếp theo: thay 8 nhân sự bằng 8 AI agents · thêm Orches coordinator · user khỏi điều phối

9 agents, 1 workflow

Bạn nói chuyện với Orches → Orches phân việc cho 8 specialist → live app



↑ *notify_orchestrator.sh "done - <artifacts>. Summary: ..."* · Orches surface kết quả + đề xuất phase tiếp

9 panes · 9 models · 1 user-facing channel (Orches)

Tại sao multi-agent?

Ba vấn đề mà single-model agent thường gặp phải

ĐẤT

Một model mạnh phải làm cả 9 việc. PM, BA, QA thực ra cần model rẻ; chỉ SA/BE/FE/Orchestrator mới cần model mạnh. Single-agent = trả giá strong-model cho mọi task.

MỜ HỒ

Subagent transcript thường bị summarize trong context. Bạn không thấy worker nghĩ gì, không reproduce được, không debug được. Audit-trail rất yếu.

LỆCH LANE

Một 'PM subagent' được gọi từ BE thường lại đi viết code. Vai trò bị blur. Không có cơ chế chặn 'stay in your lane' — chất lượng giảm dần.

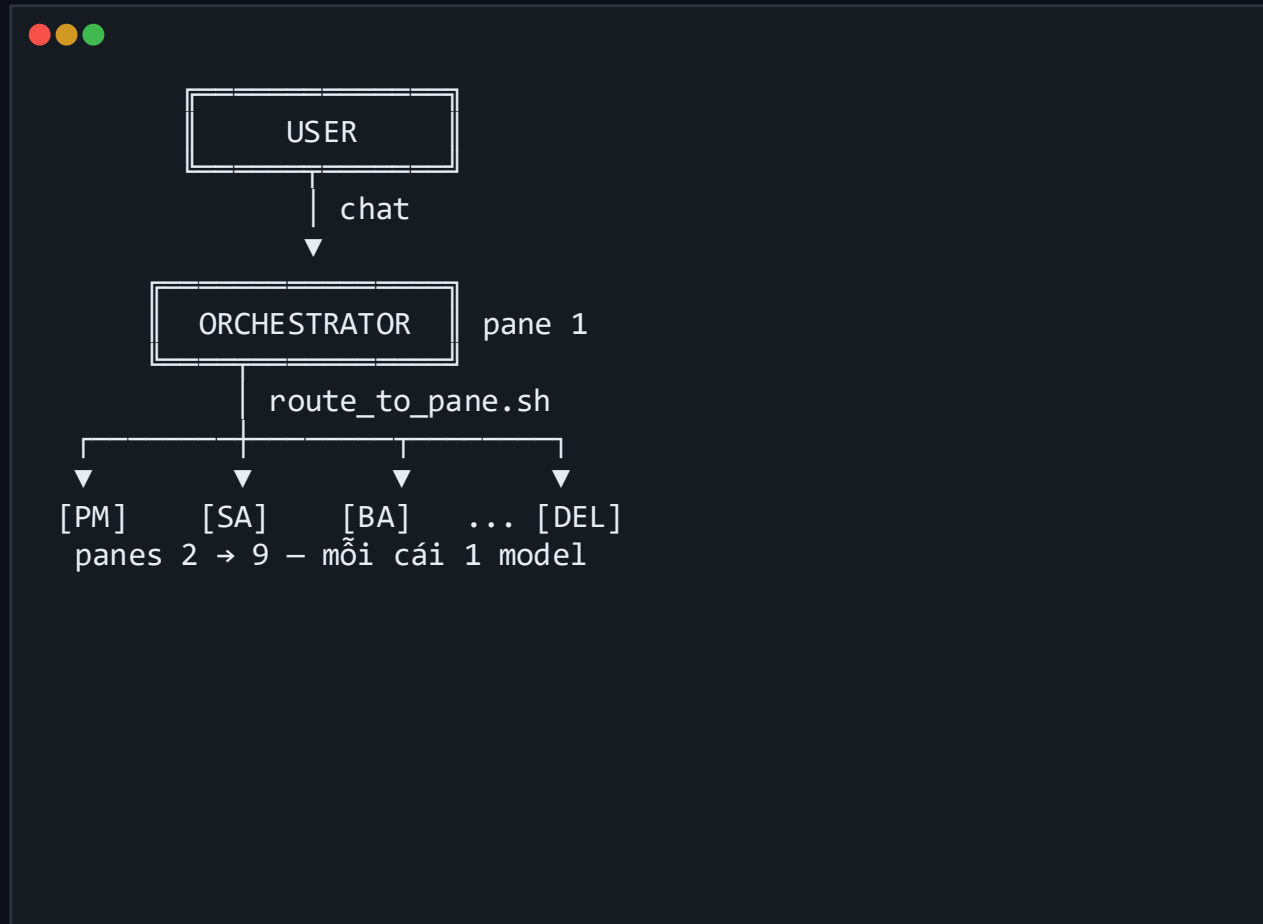
Tổng quan giải pháp

1 người dùng → 1 Orchestrator → 8 worker agent, mỗi cái 1 model riêng

Mỗi role là một tmux pane riêng, chạy engine (OpenCode hoặc Claude Code) với một model phù hợp với năng lực và chi phí. Bạn chỉ chat với Orchestrator; Orchestrator route công việc bằng shell commands thật, không dùng subagent ảo.

3 tính chất cốt lõi

- ▶ Pane = execution boundary (process thật, model thật)
- ▶ Routing = bash command thật, audit-able qua .pane_logs/
- ▶ Memory = file system, cross-session, commit vào git



BMAD là gì?

Breakthrough Method for Agile AI-Driven Development

B

Breakthrough

M

Method for Agile

A

AI-Driven

D

Development

Ý tưởng cốt lõi: chia một dự án phần mềm thành các vai trò chuyên biệt giống một đội Agile thật (PM, Architect, BA, Dev, QA, ...), và để mỗi vai trò chạy bởi một agent AI riêng với context và prompt được thiết kế kỹ. AI không thay thế Agile — AI thực thi Agile.

1. Agentic Planning

Planning artifacts (PRD, Architecture, Stories) do các agent chuyên biệt sinh ra — không bắt một LLM làm tất.

2. Context Engineering

Mỗi agent chỉ được nhận đúng context nó cần — không feed cả repo. Tiết kiệm token, tăng độ chính xác.

3. Doc-as-Spec

Tài liệu (PRD, ADR, BACKLOG...) là source of truth giữa các agent, không phải conversation history.

3 cách tiếp cận – chọn cái nào?

Single-agent vs nhiều LLM không chung context vs BMAD multi-agent

SINGLE-AGENT

1 LLM, 1 context

e.g. ChatGPT / Claude cho mọi việc

- X 1 LLM làm cả PM, SA, BA, BE, FE, QA
- X Mọi quyết định trộn trong 1 conversation
- X Không role boundary — model tự pick lane
- X Trả giá strong-model cho mọi task
- X Context đầy nhanh → hallucination, drift
- X Subagent ảo → mất audit trail

MULTI-LLM, NO CONTEXT

Nhiều tool, user relay tay

e.g. ChatGPT plan → Cursor code → Claude review

- △ Mỗi tool 1 model phù hợp việc (good)
- X User phải copy/paste context giữa tool
- X Mỗi tool có context riêng, không thấy nhau
- X Không persistent memory giữa các tool
- X Resume hôm sau: re-explain từ đầu
- X Audit = chat history rời rạc, khó replay

BMAD MULTI-AGENT

9 role × 9 model, file-based handoff

e.g. Framework này (v10.12)

- ✓ 1 agent / role, prompt + model riêng
- ✓ Handoff qua file system (PRD, ADR...)
- ✓ Routing là shell command — replay-able
- ✓ Memory cross-session, commit git
- ✓ Role boundary cứng (Stay in your lane)
- ✓ Phase gate + quality gate enforce

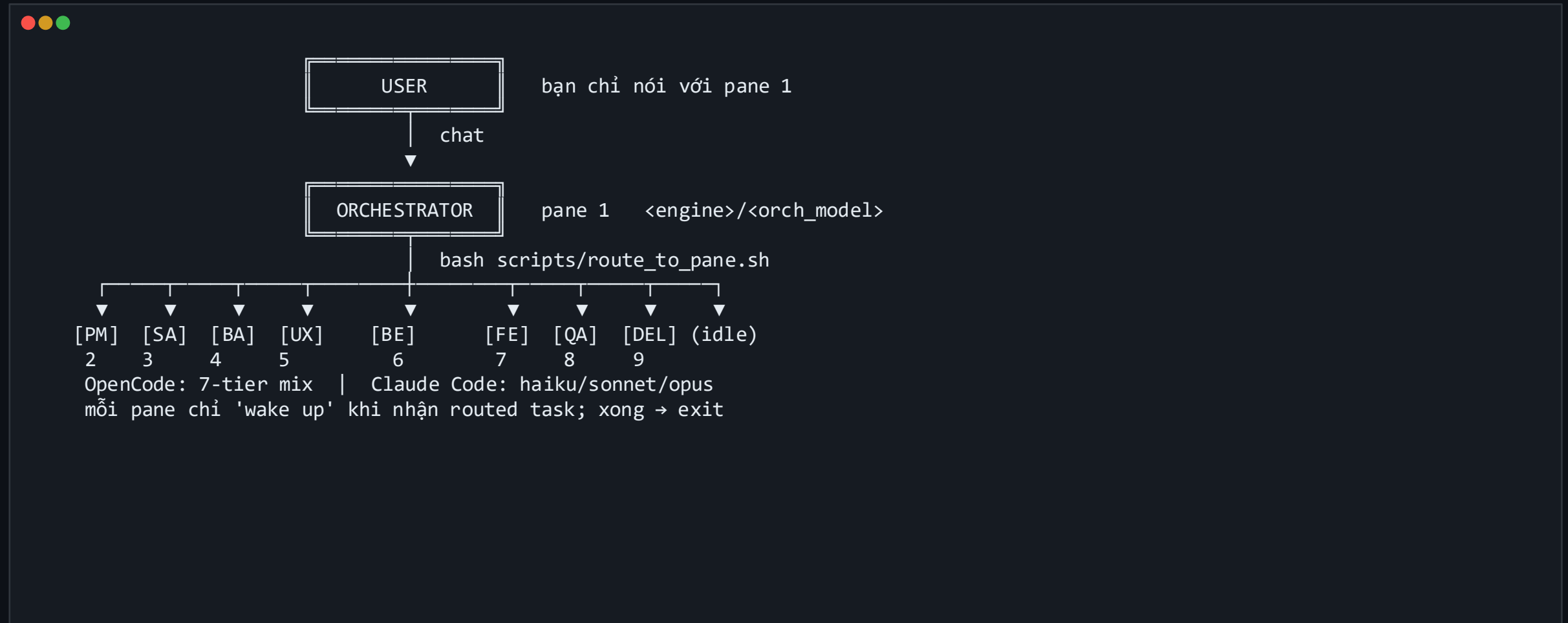
Framework hiện thực BMAD như thế nào?

Mỗi nguyên lý BMAD ánh xạ vào một cơ chế cụ thể

Agentic Planning	9 tmux pane × 9 prompt × 9 model — mỗi role có file prompts/agents/<ROLE>.md riêng
Context Engineering	Lean role-specific read list (v10.6) + memory/<ROLE>.md (v10.5) — đọc đúng cái cần
Doc-as-Spec	PRD, ADR, USER_STORIES, TEST_REPORT là source of truth — workers handoff qua file, không qua chat history
Process discipline	Phase gates v10.9, Stay-in-lane v10.10, Tech Stack Protocol v10.7
Bug-free release	VERDICT: PASS gate v10.3, BUG_REPORT.md status convention, prerequisite check v10.8
Continuity	memory/_PROJECT_STATE.md + session resume + rescan_project.sh (v10.5)

Kiến trúc tmux 9 pane

Pane 1 chat với bạn, pane 2-9 chờ được route việc



Tmux 3-window layout (v10.12)

Window 0 OC · Window 1 DESIGN · Window 2 DEV — switch bằng Ctrl+B 0/1/2



Lợi: Orchestrator full size cho chat. DESIGN/DEV tách rõ — bạn theo dõi đúng role group cần ở mỗi phase.

9 vai trò trong đội

Mỗi role có prompt + model + write boundary riêng

ROLE	TRÁCH NHIỆM CHÍNH	DEFAULT MODEL (opencode)	WRITE BOUNDARY
ORCHESTRATOR	Routing + phase gates	glm-5.1	TASK.md, _PROJECT_STATE.md
PM	Scope, PRD, MVP	deepseek-v4-flash	docs/product/PRD,ROADMAP, planning/BACKLOG
SA	Architecture, stack	deepseek-v4-pro	docs/architecture/*
BA	Business rules, stories	qwen3.6-plus	docs/business/*
UX	User flow, wireframes	mimo-v2.5	docs/product/UX_FLOW, WIREFRAMES
BE	Backend code + tests	deepseek-v4-pro	backend/*, planning/BE_PLAN
FE	Frontend code + tests	kimi-k2.6	frontend/*, planning/FE_PLAN
QA	Test plan + verdict	qwen3.5-plus	docs/qa/*, reports/*
DELIVERY	Docker, deploy, URL	glm-5	Dockerfile, compose, docs/delivery/*

Tại sao mix model = tiết kiệm?

PM/BA không cần model mạnh; SA/BE/FE cần. Một mô hình ≠ một mức giá hợp lý cho mọi việc.

FREE / CHEAP

mimo-v2.5
glm-5

Vai trò:

UX, DELIVERY

MEDIUM

deepseek-flash
qwen3.6

Vai trò:

PM, BA, QA

STRONG

deepseek-v4-pro
kimi-k2.6

Vai trò:

SA, BE, FE

Cấu hình tại config/agent_models.env — đổi model bất cứ lúc nào không sửa code.

Cơ chế routing

Orchestrator giao việc cho worker bằng bash thật, không subagent ảo

Mỗi lần Orchestrator gọi route_to_pane.sh:

- 1 Validate role + tìm pane id trong .agent_panes
- 2 Sinh task file .pane_tasks/<ROLE>_<ts>.md
- 3 Ghi receipt vào .pane_logs/_routing_receipts.log
- 4 tmux send-keys vào pane đích: ``<engine> run --model <X> "<task>"`` (theo .agent_session)
- 5 Pane đích spawn engine, đọc context, làm việc, exit

→ route_to_pane.sh REFUSES target=ORCHESTRATOR (v10.0) để tránh self-kill TUI.



```
# Bạn nói với Orchestrator:  
# "Bắt đầu phase discovery"
```

```
$ bash scripts/delegate_phase.sh \  
    discovery
```

```
[Orch] routes PM, BA, UX in parallel  
[Orch] writes 3 task files  
[Orch] tmux send-keys to panes 2,4,5
```

```
# Verify routing thật sự đã chạy:  
$ bash scripts/verify_routing.sh
```

Vòng lặp clarification

Worker không chat trực tiếp với user — hỏi qua Orchestrator



```
[SA pane]  gặp quyết định cần user (Postgres vs Mongo?)
           $ bash scripts/ask_orchestrator.sh SA "<question>"
           ↓ writes .pane_questions/SA_<ts>.md
           ↓ tmux send-keys [INBOX] vào pane 1 + Enter (v10.4)
[Orch]     auto-wake → list_pending_questions.sh → present cho USER
[USER]     "Postgres on Supabase free tier"
[Orch]     $ bash scripts/answer_role.sh SA <qid> "<answer>"
           ↓ writes .pane_answers/SA_<qid>.md
           ↓ auto-routes SA với answer attached
[SA pane]  resume — đọc answer, tiếp tục task, ghi memory/SA.md
```

Stop-and-ask threshold (universal – v10.7 + v10.10)

Vòng lặp notification

Worker báo sự kiện ngược lại Orchestrator (không cần trả lời)

Khác với ask_orchestrator.sh (cần trả lời), notify_orchestrator.sh là one-way event:

✓ Tests PASS – auto-route DELIVERY

QA chạy test xong, VERDICT: PASS:

```
$ notify_orchestrator.sh QA \  
  "Tests PASS – routing DELIVERY"  
$ route_to_pane.sh DELIVERY "..."
```

Orch auto-wake, kể user, watch deploy

App is live – URL ready

DELIVERY xong docker compose up:

```
$ notify_orchestrator.sh DELIVERY \  
  "App live – http://localhost:3000"
```

Orch nhận [INBOX], next turn:

```
→ cat docs/delivery/RUNNING_APP.md  
→ user gets URL
```

Inbox xem qua: \$ bash scripts/list_pending_questions.sh (hiển thị cả questions + notifications)

Memory layer – ký ức xuyên session

Workers chạy one-shot rồi exit. Memory file thay thế conversation history.

Mỗi role có file memory/<ROLE>.md — append-only scratchpad của decisions, conventions, gotchas. Commit vào git. Đọc đầu mỗi task, append cuối mỗi task.



```
memory/
├── README.md                format + discipline
├── _PROJECT_STATE.md        team snapshot (Orchestrator owns)
├── ORCHESTRATOR.md          routing decisions
├── PM.md    SA.md    BA.md  per-role: Decisions / Conventions /
├── UX.md    BE.md    FE.md  Gotchas / Open items I'm tracking
├── QA.md    DELIVERY.md

```

Cấu trúc entry:
2026-05-14 18:30 – Postgres on Supabase free tier
Chose Postgres because user wants free Supabase tier (.pane_answers/SA_xxx.md).

Memory ≠ docs canonical. ADR ghi decisions kiến trúc; memory ghi 'connective tissue' (lý do, gotcha, todo ngắn hạn).

Session resume – như team quay lại ngày mới

start_agents_tmux.sh tự detect RESUME vs FRESH



```
# Day 2: bạn chạy lại session
$ bash scripts/start_agents_tmux.sh \
  nestfi-ai-room

[v10.5] RESUME mode –
  non-empty memory/PM.md detected
[v10.5] Orchestrator sẽ chạy
  scripts/rescan_project.sh
  khi first turn

# 8 giây sau khi opencode load,
# auto-ping vào pane 1:
[RESUME] Session resumed – run
  bash scripts/rescan_project.sh first.
```

Orchestrator's first action

- ▶ Đọc memory/_PROJECT_STATE.md
- ▶ Tail 30 dòng memory/<ROLE>.md mỗi role
- ▶ Liệt kê docs có content (>200B)
- ▶ Đếm source code BE/FE
- ▶ Đọc RUNNING_APP.md → live URL
- ▶ Show inbox: pending Q + notifs
- ▶ Greet user: 3-5 bullet recap, đờn direction

Orchestrator KHÔNG auto-advance phase. Project tiếp tục đứng nơi để lại.

Phase pipeline – 7 giai đoạn

Giống Agile thật: discovery → design → backlog → planning → build → test → delivery

0	DISCOVERY	PM + BA + UX	PRD, BUSINESS_REQ, USER_STORIES, UX_FLOW
1	SOLUTION	SA	SOLUTION_ARCH, TECH_STACK 🚧 confirmed, ADR, API_CONTRACT
2	BACKLOG	PM + BA + UX + SA	BACKLOG ưu tiên, USER_STORIES có AC
3	PLANNING	BE + FE + QA + DEL	BE_PLAN, FE_PLAN, TEST_PLAN, DELIVERY_PLAN
4	BUILD	BE + FE	backend/* + frontend/* (source thật)
5	TEST & FIX	QA + BE + FE	TEST_REPORT VERDICT: PASS, BUG_REPORT clean
6	DELIVERY	DELIVERY	docker-compose, Dockerfile, RUNNING_APP.md + live URL

Phase gates – không nhảy phase

Không thể vào phase N khi phase N-1 chưa qua exit criteria

```
●●●
# Check một phase
$ bash scripts/check_phase_gate.sh \
  1_SOLUTION_DESIGN

# Check tất cả phase tới đó
$ bash scripts/check_phase_gate.sh \
  --through 3_IMPLEMENTATION_PLANNING

# Officially advance (refuse nếu gate fail)
$ bash scripts/advance_phase.sh \
  4_BUILD

# Override khi user explicit waive:
$ ADVANCE_PHASE_FORCE=1 \
  bash scripts/advance_phase.sh 6_DELIVERY
```

Semantic checks (vượt khỏi file-exists)

- ▶ Phase 1 cần `TECH_STACK.md` có "Confirmed by user"
- ▶ Phase 4 cần `backend/` + `frontend/` có source files thật
- ▶ Phase 5 cần `VERDICT: PASS` + no `OPEN_CRITICAL` / `OPEN_MAJOR` / `RETEST_FAIL`
- ▶ Phase 6 cần `RUNNING_APP.md` có URL `http(s)://` thật

Release gate – bug-free hoặc no-ship

DELIVERY refuse deploy nếu TEST_REPORT FAIL hoặc BUG_REPORT có blocker

Bug status convention (QA)

● OPEN_CRITICAL	Blocks all use of the app
● OPEN_MAJOR	Affects primary user story
● OPEN_MINOR	Cosmetic / non-critical
● RETESTING	Fix pushed, QA verifying
● RETEST_PASS	Fix verified
● RETEST_FAIL	Fix didn't work
● FIXED	Closed
● WONT_FIX	Accepted, not fixing



```
# DELIVERY prereq check (v10.9):  
$ bash scripts/check_prerequisites.sh \  
    DELIVERY  
  
# Hard-blocks unless ALL true:  
reports/TEST_REPORT.md exists  
_SIGNED_TECH_STACK_    (Confirmed by user)  
_VERDICT_PASS_         (TEST_REPORT)  
_NO_OPEN_CRITICAL_MAJOR_ (BUG_REPORT)  
_CODE_BE_              (backend/ has src)  
_CODE_FE_              (frontend/ has src)  
  
→ DELIVERY tự refuse, notify Orch,  
   Orch route BE/FE/QA fix trước.
```

Requirements – máy bạn cần gì?

macOS + tmux + 1 trong 2 engine (opencode-go HOẶC Claude Code); Linux OK, Windows cần WSL2

macOS

PRIMARY

Đã test kỹ. Cần macOS 12+ (Monterey).

Linux

WORKS

Cần chỉnh nhẹ stat-cmd (BSD vs GNU). Đã handle trong verify_routing.sh.

Windows

WSL2 only

Pure Windows (Git Bash, MSYS) chưa test. Khuyến nghị WSL2 + Ubuntu 22.

REQUIRED – bắt buộc

tmux	≥ 3.0 (pane_current_command, display-message -p)
bash	≥ 3.2 (macOS default OK)
git	clone repo + lưu memory/ vào VCS
engine	opencode-go HOẶC claude (chọn 1, default: opencode)
Internet	engine gọi model API qua mạng
Models	9 model trong config/agent_models.env phải accessible

OPTIONAL – theo phase / stack

Docker	Phase 6 DELIVERY: docker compose up
Python ≥ 3.12	Nếu SA chọn FastAPI/Django stack + pytest
Node ≥ 20	Nếu SA chọn Next/Vite/TS stack + vitest
curl, jq	DELIVERY probe health endpoint, parse JSON config
ffmpeg/sox	Nếu QA cần media processing trong test
Node ≥ 18	Chỉ cần nếu bạn muốn rebuild slide deck này

Verify nhanh: `$ tmux -V && (opencode --version || claude --version) && git --version && bash --version`

Engine choice (v10.12)

Chọn CLI engine khi start session: `--engine opencode (default)` hoặc `--engine claude`

OpenCode (default)

for: newbie / experimental

```
$ bash scripts/start_agents_tmux.sh <project>
```

Multi-provider, mix 7 tier model (deepseek, qwen, kimi, glm, mimo...). Tốt để thử nghiệm rẻ.

ORCH	opencode-go/glm-5.1
PM/BA	opencode-go/deepseek-flash, qwen3.6
SA/BE	opencode-go/deepseek-v4-pro
FE	opencode-go/kimi-k2.6
QA	opencode-go/qwen3.5-plus

Config: `.opencode/config.json + opencode.json`

Check : dynamic (gọi ``opencode models``)

Claude Code

for: pro / single-vendor

```
$ bash scripts/start_agents_tmux.sh <project> --engine claude
```

Anthropic only, 3 tier (haiku/sonnet/opus). Setup nhỏ gọn — 1 API key duy nhất.

ORCH	claude-sonnet-4-6
PM/BA	claude-haiku-4-5
SA/BE	claude-opus-4-6
FE	claude-sonnet-4-6
QA/DEL	claude-haiku-4-5

Config: `.claude/settings.json`

Check : static (whitelist trong `claude.env`)

Engine choice được persist vào `.agent_session` — tất cả script (`route_to_pane`, `answer_role`, ...) đọc tự động.

Quick start – 5 lệnh để khởi động

Clone từ GitHub → bootstrap project → talk to Orchestrator



```
# 1. Clone từ GitHub (monorepo learning-vault)
$ git clone https://github.com/minhtuan-labs/learning-vault.git
$ cd learning-vault/frameworks/product-engineering-template

# 2. Copy template ra project mới (hoặc dùng tại chỗ)
$ cp -R . ~/my-project && cd ~/my-project
$ chmod +x scripts/*.sh

# 3. Mô tả ý tưởng – đây là input duy nhất của user
$ $EDITOR PRODUCT_IDEA.md

# 4. Pick engine + verify models (v10.12)
$ bash scripts/check_models.sh opencode # hoặc: claude

# 5. Khởi động 9-pane / 3-window tmux session
$ bash scripts/start_agents_tmux.sh my-project # default opencode
# hoặc: bash scripts/start_agents_tmux.sh my-project --engine claude
```

Sau đó chỉ nói với pane 1 (Orchestrator). 8 pane còn lại tự awake khi cần.

Workflow – Day 1 (Discovery → Solution)

Bạn chỉ chat với Orchestrator, mỗi turn ngắn gọn



USER : Bắt đầu dự án NestFi – app quản lý tài chính gia đình.

ORCH : (delegate_phase.sh discovery → PM, BA, UX song song)
Đã route 3 agent. Sẽ ping khi có câu hỏi hoặc xong.

[5 phút sau, SA chưa cần – PM file question vì PRODUCT_IDEA chưa đủ]

[INBOX] new clarification from PM – đối tượng user chính ai? B2C? B2B?

ORCH : PM hỏi: bạn nhắm user B2C (cá nhân/gia đình) hay B2B (kế toán)?

USER : B2C – gia đình 2-5 thành viên, không cần kế toán pro.

ORCH : (answer_role.sh PM ... → PM resume → ghi PRD)

USER : OK tiếp tục solution design.

ORCH : (delegate_phase.sh solution → SA, ask về 12 stack dimensions)

SA hỏi: Stack đề xuất – Next.js / FastAPI / Postgres / Tailwind ...

USER : OK với defaults, dùng Supabase free tier.

ORCH : (answer_role.sh SA ... → SA viết TECH_STACK, ADR, API_CONTRACT)

Workflow – Day 2+ (Build → Test → Release)

Resume từ memory, đẩy tiếp phase, ship

```
●●●
$ bash scripts/start_agents_tmux.sh nestfi-ai-room
[v10.5] RESUME mode – non-empty memory/SA.md detected

[Orch auto-ping] [RESUME] Run rescan_project.sh first.
ORCH : Chào buổi sáng. Phase hiện tại: 3_PLANNING (BE+FE+QA+DEL done).
       Sẵn sàng vào 4_BUILD. Có 0 question pending. Tiếp tục?
USER : Yes, build.
ORCH : (delegate_phase.sh build → BE + FE)

[~30 phút sau BE và FE đều notify done]
USER : Ship it.
ORCH : (delegate_phase.sh release → test phase first)
       QA chạy test, viết VERDICT: PASS → auto-route DELIVERY.
       DELIVERY hỏi: ports FE=3000 BE=8000 DB=5432 OK?
USER : OK defaults.
ORCH : (answer_role.sh DELIVERY → docker compose up --build -d → URL ready)
       App live tại http://localhost:3000  ▣  RELEASE_NOTES updated.
```

Tech Stack Confirmation Protocol (v10.7)

SA không được tự pick Next.js/React/Postgres trên auto-pilot

Trước khi viết TECH_STACK.md, SA phải gửi 1 câu hỏi batched với 12 dimensions để user confirm hoặc override:

1. Frontend framework

2. Frontend language

3. UI / design system

4. Backend language

5. Backend framework

6. Primary datastore

7. ORM / data layer

8. Auth approach

9. Cache / queue / search

10. Hosting target

11. Code layout

12. Package manager

Sau khi user trả lời, SA quote answer verbatim vào TECH_STACK.md + dòng "Confirmed by user (qid: SA_xxx)". Phase 1 gate kiểm tra dòng này.

Stay in your lane (v10.10)

Mỗi role có write boundary. Cross-boundary = route owner, không tự làm.

PM	docs/product/{PRD,ROADMAP}.md, planning/{BACKLOG,OPEN_QUESTIONS}.md
SA	docs/architecture/*.md
BA	docs/business/*.md
UX	docs/product/{UX_FLOW,WIREFRAMES,DESIGN_NOTES}.md
BE	backend/* (source), planning/BE_PLAN.md
FE	frontend/* (source), planning/FE_PLAN.md
QA	docs/qa/*.md, reports/*.md, plus tests if BE/FE didn't
DELIVERY	Dockerfiles, docker-compose.yml, .env.example, docs/delivery/*
ORCHESTRATOR	TASK.md, memory/_PROJECT_STATE.md (coordinator only)

Độc tự do, ghi giới hạn. DELIVERY hit FE build bug → route FE (hoặc SA triage), KHÔNG tự sửa frontend/.

Prerequisite check (v10.8)

Worker không bao giờ guess thiếu input — block và route upstream



Worker chạy đầu mọi task:

\$ bash scripts/check_prerequisites.sh BE

Prerequisite check for BE

docs/architecture/API_CONTRACT.md	MISSING
docs/architecture/TECH_STACK.md	OK
planning/BE_PLAN.md	SKELETON

BLOCKED — BE cannot proceed. 2 missing.

Required action:

notify_orchestrator.sh BE \
"Cannot proceed — missing API_CONTRACT.md,
BE_PLAN.md. Need SA, BE to produce."

Orchestrator coordinates

1. Nhận notification BE blocked.
2. Parse missing list + owners.
3. Route mỗi upstream owner:
→ SA: produce API_CONTRACT.md
4. Update memory/_PROJECT_STATE.md
Active workstreams = SA writing
5. Khi SA notify "done":
check_prerequisites.sh BE → OK
route_to_pane.sh BE "<orig task>"

Build failure routing (v10.10)

DELIVERY phát hiện lỗi build → route đúng owner, KHÔNG tự sửa



Anti-pattern (v10.10 forbids):

```
[DELIVERY] docker compose build → fails: "HStack.justify"
            → reads frontend/nestfi/pages/family_selection.py    X WRONG
            → greps "justify" in frontend/nestfi                 X WRONG
            → edits FE source                                     XX VERY WRONG
```

Correct flow (v10.10):

```
[DELIVERY] tail -50 docs/delivery/_last_build.log
            → "target frontend: failed to solve"
            $ route_to_pane.sh FE "Build failed. See _last_build.log.
              Failing line: HStack.justify. Likely outdated Reflex
              API in pages/. Fix FE source. Notify when ready."
            $ notify_orchestrator.sh DELIVERY "Build FAIL – routed FE"
            → exit

[Orch]      ack user → wait FE notify done → re-route DELIVERY
```

Framework files immutable (v10.11)

Agents không sửa AGENTS.md, prompts/, scripts/, config/ ... Sync template bằng script.

Read-only cho mọi agent

```
AGENTS.md
README.md  VERSION.md  LICENSE  .gitignore
PRODUCT_ENGINEERING.md
.opencode/config.json  opencode.json
scripts/*.sh           (tàn bộ)
prompts/agents/*.md    (mọi role prompt)
config/*               (env, json, md)
planning/{PANE_ROUTING, ORCH_RUNTIME, AGENT_WORKFLOW}.md
docs/delivery/{OPENCODE_SETUP, TMUX_USAGE}.md
memory/README.md
```

Sync an toàn từ template mới

```
# Dry-run xem sẽ làm gì:
$ bash scripts/sync_framework_from_template.sh \
  ../product-engineering-template --dry-run

# Apply:
$ bash scripts/sync_framework_from_template.sh \
  ../product-engineering-template
```

Mọi file bị overwrite được backup vào
`.framework_sync_backup/<timestamp>/`

PRODUCT_IDEA.md, TASK.md, memory/*, docs/
outputs, backend/, frontend/ KHÔNG bị đụng.

Continuous orchestration (v10.13/14)

Workers tự báo cáo, watcher tự reroute — không cần user nhắc

v10.13 – Complete-and-notify

Mọi worker (PM/SA/BA/UX/BE/FE/QA/DEL) MUST call:

```
bash scripts/notify_orchestrator.sh \  
  <ROLE> "Done – <paths>. Summary: <2-3 sentences>"
```

trước khi exit – kể cả khi 'rõ ràng đã xong'.

v10.18 thêm safety net: route_to_pane.sh wrapper
tự fire fallback notify nếu worker quên
(dedup 30s window – không double-fire).

Active reporting: Orchestrator mỗi turn chạy
list_pending_questions.sh + list_pending_watches.sh
+ check_phase_gate.sh → surface trước khi reply user.

v10.14 – watcher_daemon

Background process – engine-agnostic.

Khởi động bởi start_agents_tmux.sh
(PID lưu .watcher.pid). Mỗi 15 giây poll:

1. .pane_watches/*.watch – workers parked, đợi upstream file landing
2. .pane_heartbeats/*.beat – workers active (touch mỗi 10s); detect stall sau 90s

Khi dependency landing:

- route_to_pane.sh <ROLE> "<resume task>"
- notify_orchestrator (auto-resume signal)

Worker chỉ cần: bash scripts/file_watch.sh
<ROLE> <MISSING_FILE> "<resume msg>"

User effort giảm 20-30% → ~5% (chỉ kick off + answer clarifications + confirm transitions)

Auto-wake – Claude Code reliability (v10.15-18.1)

3 delivery methods + verify + visible fallback — Ink TUI compatible

`scripts/_ping_orchestrator_pane.sh:`

Method 1 – `tmux load-buffer + paste-buffer -p` (bracketed paste, fast)

tmux-native bracketed paste signalling. Works for OpenCode BubbleTea + most TUIs.

Method 2 – `tmux send-keys -l "$msg"` (literal characters, fast)

Fallback if method 1 doesn't deliver. Sends raw chars without paste-mode wrapping.

Method 3 – `typewriter (char-by-char, 40ms)` (slow but reliable)

Mimics real human typing – bypasses any rapid-input filtering in Claude's Ink TUI.

Verify each method

tmux capture-pane → grep 30-char sentinel.
Pass → send Enter (x2 – Claude eats first sometimes).
Fail → try next method.

If all 3 fail – visible fallback

tmux display-message toast + macOS osascript notification (Glass sound). Plus Prefix+W binding calls wake_orchestrator.sh manually.

v10.18.1 — pane_current_command='2.1.143' (Claude on macOS) → permissive case statement

--free overlay (v10.16)

Iterate workflow without burning credits — overlay file ghi đè *_MODEL

Usage:

```
bash scripts/start_agents_tmux.sh <project> --engine opencode --free
```

```
bash scripts/start_agents_tmux.sh <project> --engine claude --free
```

config/engines/opencode-free.env

Default: opencode/big-pickle
(confirmed working in opencode-go fork)

9 roles → cùng 1 model.
Workflow correctness không cần đa
dạng model, chỉ cần model chạy được.

Override: OPENCODE_FREE_MODEL=...
Discovery: bash scripts/list_free_models.sh

config/engines/claude-free.env

9 roles → claude-haiku-4-5
(cheapest tier — Claude không có \$0 tier)

Nếu Pro/Max subscription:
haiku usage tính quota, không metered.

Cảnh báo: ANTHROPIC_API_KEY set →
start_agents_tmux.sh refuses launch
(override với CLAUDE_API_MODE_ACK=1).

FREE_MODE persisted vào .agent_session — route_to_pane.sh / check_models.sh tự honor

Hard lane discipline (v10.18/19)

3-layer defense — Orchestrator điều phối, không executive — blocked tại shell level

Layer 1 – Prompt (soft)

ORCHESTRATOR.md: 'YOU ORCHESTRATE – YOU DO NOT EXECUTE' (action ownership table).
'Bug Report Triage Protocol' – user báo bug → MUST route QA → QA writes regression test
→ identify owner (BE/FE/DELIVERY) → route fix → QA retest → DELIVERY redeploy.

Layer 2 – PATH guard (HARD)

scripts/guards/_block_orchestrator.sh + auto-generated wrappers cho ~25 commands:
docker, docker-compose, kubectl, helm → DELIVERY
psql, mongo, mysql, redis-cli, alembic → BE
python, node, pip, npm, yarn, poetry → BE/FE

Chỉ Orchestrator pane có scripts/guards/ trong PATH. Worker panes giữ PATH normal.

Layer 3 – Audit

scripts/check_lane_violations.sh: tail PATH-guard log + inspect Orchestrator descendant processes (flag engineering binaries running mid-turn).

BLOCKED – Orchestrator cannot execute engineering commands. Route to DELIVERY instead.

DELIVERY Port Configuration Protocol (v10.20)

Scan trước, ask user 3 options sau — không silent pick

1. Scan defaults

```
bash scripts/check_port_conflicts.sh 3000 8000 5432  
→ PORT=3000 STATUS=FREE | PORT=5432 STATUS=TAKEN BY="docker:postgres-other"
```

2. Ask user – 3 options

(A) Preferred: user trả lời 'FE=X BE=Y DB=Z' → DELIVERY re-scan để xác nhận
(B) Auto: user trả lời 'auto' → DELIVERY chạy --suggest 3 → pick first FREE
(C) Defaults: user trả lời 'defaults' → chỉ valid nếu scan ban đầu all FREE

Plus: 'DB public hay internal?' → security exposure choice

3. Persist + document

```
.env.example (FE_PORT=, BE_PORT=, DB_PORT=) + docker-compose.yml ports:  
+ docs/delivery/DELIVERY_PLAN.md 'Port assignments' section (audit trail).
```

Eliminates 'docker compose up red because 5432 was taken' class of failure

Troubleshooting cheat sheet

Những lỗi phổ biến và cách thoát nhanh

Pane 1 stuck ở dquote>

Ctrl+C → auto-relaunch loop sẽ prompt Enter để vào lại TUI

Orch nói routed nhưng không thấy

\$ bash scripts/verify_routing.sh – nếu trống, Orch chỉ chat, chưa thực sự bash

Có Q pending mà Orch im lặng

Gõ 'có câu hỏi gì pending không?' – Orch chạy list_pending_questions.sh

DELIVERY refuse deploy

Đọc reports/TEST_REPORT.md + BUG_REPORT.md – fix blocker rồi re-route

Build fails – DELIVERY tự sửa FE

Stop. Re-route DELIVERY với hint follow Build Failure Routing Protocol

Model not found

\$ bash scripts/check_models.sh <engine> – fix config/engines/<engine>.env

memory/ drift khỏi reality

\$ bash scripts/rescan_project.sh – Orch update memory/_PROJECT_STATE.md

Version history

Từ ý tưởng cho tới v10.23 — mỗi bước thuần additive, không phá rule cũ

v10.0	Real config + Task subagent disabled + verify_routing	v10.11	Framework files immutable + safe sync script
v10.1	Drop --config flag; AGENTS.md autoload; kill racy auto-paste	v10.12	Engine choice (OpenCode Claude) + 3-window tmux layout
v10.2	Clarification loop (ask/answer/list) + Orch auto-relaunch	v10.13	Complete-and-notify mandate + Active reporting
v10.3	QA verdict gate; DELIVERY ships end-to-end; notify_orchestrator	v10.14	Continuous orchestration: watcher_daemon auto-resume
v10.4	Auto-wake Orchestrator via [INBOX] keystroke ping	v10.15	Bracketed-paste auto-wake for Claude's Ink TUI
v10.5	Durable memory/ layer + session resume + rescan_project	v10.16	--free model overlay (engine-aware, FREE_MODE persisted)
v10.6	Lean role-specific read list (cut token waste ~50%)	v10.17	ACT-DO-NOT-ANNOUNCE rule for verbose models
v10.7	Tech Stack Confirmation Protocol (SA + BE/FE addenda)	v10.18	Safety nets: silent-exit notify + heartbeat + lane lint
v10.8	Prerequisite check — fail fast on missing inputs	v10.19	Hard PATH guards + Bug Report Triage Protocol
v10.9	Phase gates + quality gates (VERDICT, no critical bugs)	v10.20	DELIVERY Port Configuration Protocol
v10.10	Stay-in-lane + Build Failure Routing Protocol	v10.21	PaneC team identity + Orches/Deli display names
		v10.22	Deli exhaustive port scan + post-deploy verify
		v10.23	Layer-4 hook blocks Orchestrator Write/Edit

Nguyên tắc: mỗi version thêm chứ không bớt. Grep verify trước và sau mỗi thay đổi.

Next update for this framework

Coming soon — đang trong giai đoạn thiết kế

Scrum Master + agile cadence

v10.12 (planned) — vai trò facilitator để team có retrospective, theo dõi impediment, và đo health của project



Retrospective

Sau mỗi phase, tổng hợp 'what went well / didn't / improve' vào docs/process/RETRO.md



Impediment log

Track Q pending lâu, bug lặp, role hay block → escalate cho Orchestrator



Health metrics

Turn / phase, Q / role, rework ratio, time-to-PASS — visibility cho user

Đang cân nhắc: SM là 1 pane riêng (pane 10) hay merge vào Orchestrator. Quyết định sau khi thử nghiệm.

Vì sao tự build framework này?

8 lý do — vì sao không dùng Claude/Cursor/CrewAI mà phải tự build

A Mỗi việc dùng đúng model

Việc khó (kiến trúc, code backend) → model mạnh.
Việc đơn giản (viết PRD, test) → model rẻ.
→ Trả đúng tiền cho đúng việc, không lãng phí.

C Giao việc bằng lệnh bash thật

route_to_pane.sh BE "..." — không phải magic call.
Lệnh nào chạy, kết quả nào, đều có log lại.
→ Xem lại được, sửa được, chạy lại được.

E Bắt đi đúng quy trình phần mềm

Discovery → Design → Build → Test → Deploy.
QA phải PASS mới được deploy, bug có report.
→ Sản phẩm chất lượng, không 'AI làm bừa'.

G Nhớ việc qua các session

Mỗi agent có file memory/<ROLE>.md commit git.
Đóng máy hôm nay, mai mở — đọc được context hôm qua.
→ Project survive crash, power loss, đổi máy.

B Mỗi agent một pane riêng

9 agents = 9 process độc lập trong tmux.
Mỗi pane có log riêng, theo dõi được ai đang làm gì.
→ Debug khi lỗi, audit khi cần — không 'AI hộp đen'.

D Đổi engine AI dễ như đổi config

OpenCode / Claude Code / AI tương lai — plug-in được.
Provider lên giá? Chuyển model rẻ hơn ngay.
→ Không bị khóa với 1 nhà cung cấp.

F Bạn vẫn là người quyết định

AI hỏi trước decision lớn: tech stack, port, fix bug.
Không guess, không tự quyết — bạn confirm rồi mới làm.
→ AI làm cho bạn, không thay bạn.

H Nhẹ, chạy 100% local

Chỉ cần tmux + bash + 1 AI CLI là đủ.
Không Python deps, không Docker, không cloud account.
→ MacBook là chạy được — vs Devin/Lovable phụ thuộc cloud.

Tiết kiệm chi phí · Quản lý dễ · Bạn vẫn quyết định · Chạy local · Không khóa nhà cung cấp

```
$ git clone github.com/minhtuan-labs/learning-vault  
$ cd learning-vault/frameworks/product-engineering-template  
$ bash scripts/start_agents_tmux.sh my-project  
$ # nói với pane 1: "bắt đầu dự án X"
```

Cảm ơn đã đọc tới đây.

Framework này là một dự án nhỏ — phản hồi, issue, PR đều rất welcome.

Đặc biệt nếu bạn thử với engine khác (aider, goose, codex, gemini-cli...) hoặc model provider khác và gặp gì lạ.

MIT License

v10.23

tmux + opencode/claude

Tuan M. Pham